Virtual Environments & Reproducibility

Why use virtual environments

A **virtual environment** isolates project dependencies so multiple Python projects don't conflict.
Example problem: Maggie (Mac OS) and John (Windows) run the same notebook but get different results because of differing library versions.
Solution: they share a `.yml` file or `requirements.txt` that precisely lists every package + version, so anyone can recreate the exact environment. Git + Conda make these environments portable and reproducible .

Benefits

- **Reproducible research** – results repeat on any machine.
- **Explicit dependencies** – every library version is pinned.
- **Improved collaboration** – teams sync environments easily.
- **Broader skill set** – industry-standard practice for ML and data science.

Virtualenv (Built-in Python tool)

- **What it is:** the native Python environment manager (`venv`/`virtualenv` modules).
- **How it works:** creates an isolated folder containing its own `site-packages` and Python interpreter.

Key commands

```
# Create
virtualenv your_env_name
# Activate
source your_env_name/bin/activate   # (Linux/macOS)
your_env_name\Scripts\activate.bat  # (Windows)
# Install packages
```

pip install package_name==version
Deactivate
deactivate

Location: folder created inside the current project directory.
When to use: lightweight local isolation for scripts or small projects.
Limitations: doesn't manage non-Python binaries (e.g., CUDA, system libs); only works cleanly within one OS type.

Conda (Anaconda/Miniconda manager)

- **What it is:** an environment + package manager that handles Python and non-Python dependencies (C, C++, CUDA etc.).

- **Where it lives:** under `/anaconda3/envs/your_env_name/`.

Commands

```
# Create environment with specific Python version
conda create --name your_env_name python=3.8
# Activate / Deactivate
conda activate your_env_name
conda deactivate
# Install packages (can mix conda and pip)
conda install numpy=1.25
pip install fastapi==0.110
# Export to YAML (for sharing)
conda env export > my_environment.yml
# Recreate from YAML
conda env create -f my_environment.yml
```

Advantages

- Works across Windows, macOS, Linux.

- Can install compiled packages (e.g., TensorFlow GPU) without system-level setup.

- Allows mixing pip + conda packages safely.

YAML & requirements.txt

Format	Usage	Command
`environment.yml`	Conda YAML file (includes channels + versions)	`conda env export > environment.yml`
`requirements.txt`	Pip-style text file	`pip freeze > requirements.txt`

Both ensure identical setups for collaborators or CI/CD pipelines.

When to choose what

Scenario	Best tool
Pure Python project (no system libs)	virtualenv
ML/DS project with CUDA or C libs	conda
Shared across OS (team work)	conda
Simple web microservice	virtualenv or venv

Real-world workflow

1. Create → Activate → Install libs.
2. Run experiments / train models.
3. Export `.yml` for teammates or CI/CD build.

4. Recreate same env anywhere using `conda env create -f`.

Quick recap (2 lines)

Virtualenv isolates Python-only projects; **Conda** manages full-stack ML environments + YAML exports for replication .
Both enable reproducibility and collaboration across machines and OSs .

MCQs (with answers + reasons)

1️⃣ **Which command creates a new virtualenv?**
A) `conda create --name`
B) `virtualenv envname`
C) `pip install envname`
D) `python init env`
✅ **B.** `virtualenv` creates the environment folder .

2️⃣ **Where does a Conda environment live by default?**
A) Project root
B) Inside Anaconda's `/envs` directory
C) User Downloads
D) /opt/local
✅ **B.** Stored under `/anaconda3/envs/your_env_name` .

3️⃣ **How to export an exact copy of a Conda environment?**
✅ `conda env export > environment.yml` .

4️⃣ **To recreate from YAML on another machine, use:**
✅ `conda env create -f environment.yml` .

5️⃣ **Virtualenv is a:**
✅ Python-native environment manager that uses `pip` for packages .

6️⃣ **Conda's advantage over virtualenv?**
✅ Manages non-Python deps (CUDA, C/C++) and works cross-platform .

7️⃣ **Which file format ensures exact versions for reproducibility?**
✅ `.yml` or `requirements.txt` files .

8️⃣ **After `conda activate env`, to exit:** ✅ `conda deactivate` .

9️⃣ **True statement:** ✅ “Conda can use both `conda install` and `pip install`.” .

🔟 **Goal of virtual environments:** ✅ Isolate project dependencies to enable reproducible results .

Short answers (2–3 sentences)

1. Why use virtual environments in ML?
They prevent library version conflicts and allow each project to run independently, ensuring reproducibility and team collaboration across different machines .

2. How does virtualenv work?
It creates an isolated folder with its own Python binary and `site-packages`, so `pip` installs go inside that sandbox and don’t affect global Python libs .

3. What makes Conda useful for ML projects?
Conda handles Python and system packages together (e.g., NumPy + CUDA), works cross-platform, and exports YAML files for exact replication .

4. Difference between `.yml` and `requirements.txt`?
YAML lists channels and dependencies for Conda; `requirements.txt` is a pip-only text list. Both guarantee reproducible installs .

5. What does `conda env export` do?
It writes all installed packages + versions to a YAML file so others can clone the environment exactly .

6. When would you prefer virtualenv over Conda?
For small, pure-Python apps or when you don’t need GPU/system libraries — it’s lightweight and built into Python itself .